```
/*

#include<stdio.h>
#include<stdlib.h>

void pur(int *a,int n)
{
        if(n==0)
        return;
        else
        {
                printf("%d ",*(a+(4*(n-1))));
                return pur(a,n-1);
        }
}

int main()
{
        int *a,n;

        printf("n : ");
        scanf("%d",&n);

        a=(int *)malloc(sizeof(n));

        printf("Input Array : ");
        for(int i=0,p=0;i<n;i++,p=p+4)
        scanf("%d",a+p);

        printf("Modified Array : ");

        pur(a,n);

        return 0;

}

*/









/*

#include<stdio.h>

int main()
{
        int n;
```

```
        printf("n : ");
        scanf("%d",&n);

        for(int i=0;i<n;i++)
        printf("* ");
        printf("\n");

        for(int i=0;i<n-2;i++)
        {
                printf("* ");
                for(int j=0;j<n-2;j++)
                printf("  ");
                printf("* ");
                printf("\n");
        }

        for(int i=0;i<n;i++)
        printf("* ");
        printf("\n");

        return 0;
}

*/




/*

#include<stdio.h>

int fib(int n)                                  //
recursion                                                                       //
FIbonacci
{
    int x=0,y=1,z;

    printf("Fib : %d ",x);

    if(n==1)
    return 1;

    printf("%d ",y);

    if(n==2)
    return 2;

    for(int i=0;i<n-2;i++)
    {
        z=x+y;
```

```
        printf("%d ",z);
        x=y;
        y=z;
    }
}

int main()
{
    int n;

    printf("n : ");
    scanf("%d",&n);

    fib(n);

    return 0;
}

*/




/*

#include<stdio.h>

void fibo(int x,int y,int n)            //Fibonacci using recursion
{
        int z;
        if(n!=0)
        {
                printf("%d ",x);
                z=x+y;
                x=y;
                y=z;
                return fibo(x,y,n-1);
        }

}

int main()
{
    int n,x=0,y=1;

    printf("n : ");
    scanf("%d",&n);

    printf("Fib :");
    fibo(x,y,n);

    return 0;
}
```

```
*/




/*

#include<stdio.h>

int main()                                  //del Kth alternate ele
{
    int n,k,z,a[50],q=1;

    printf("n : ");
    scanf("%d",&n);

    for(int i=0;i<n;i++)
    scanf("%d",&a[i]);

    printf("k : ");
    scanf("%d",&k);

    for(int i=0;i<n-1;i++)
    a[i]=a[i+1];

    z=n-1;

    for(int i=0;i<n/k;i++)
    {
        for(int j=q*(k-1);j<z;j++)
        a[j]=a[j+1];

        q++,z--;
    }

    printf("Output : ");
    for(int i=0;i<z;i++)
    printf("%d ",a[i]);

    return 0;
}

*/




/*
```

```
#include<stdio.h>

int main()
{
        int t=2,z,s;

        for(int i=3;i<75;i++)
        {
                z=0;
                for(int j=2;j<i;j++)
                {
                        if(i%j==0)
                        {
                                z++;
                                break;
                        }

                }
                if(z==0)
                {
                        s=i-t;
                        if(s==2)
                        printf("%d %d\n",t,i);
                }
                t=i;
        }

        return 0;
}

*/




/*

#include<stdio.h>

int main()
{
        int n=30,c=0;

        printf("Prime : ");
        for(int i=1;i<=n;i++)
        {
                c=0;
                for(int j=2;j<=i/2;j++)
                {
                        if(i%j==0)
                        c++;
                }
```

```
                if(c==0)
                printf("%d ",i);
        }


        return 0;
}

*/



/*

#include<stdio.h>

int main()                                                      //Twin Prime
{
        int n=100,c=0,t=2;

        printf("Twin Prime : ");
        for(int i=1;i<=n;i++)
        {
                c=0;
                for(int j=2;j<i;j++)
                {
                        if(i%j==0)
                        c++;
                }
                if(c==0)
                {
                        if(i-t==2)
                        printf("(%d,%d)   ",t,i);
                        t=i;
                }
        }
        printf("\n");


        return 0;
}

*/




/*

#include<stdio.h>
#include<stdlib.h>

#define MAX 10
```

```
typedef struct                                                  //stack
{
        int data[MAX];
        int top;
}stack;


void initial(stack *s)
{
        s->top = -1;
}


int empty(stack *s)
{
        return s->top == -1 ;
}


int full(stack *s)
{
        return s->top == MAX-1;
}


void push(stack *s, int x)
{
        if(full(s))
        printf("Overflow\n");
        else
        s->data[++s->top]=x;
}


int pop(stack *s)
{
        if(empty(s))
        {
                printf("underflow\n");
                return -1;
        }
        else
        return s->data[s->top--];
}


int peek(stack *s)
{
        if(empty(s))
        {
                printf("Empty\n");
                return -1;
        }
```

```
        else
        return s->data[s->top];
}


int main()
{
        stack s;

        initial(&s);

        push(&s,100);
        push(&s,200);
        push(&s,300);
        push(&s,400);


        printf("%d\n",peek(&s));

        printf("%d\n",pop(&s));

        printf("%d\n",pop(&s));

        printf("%d\n",peek(&s));

        printf("%d\n",pop(&s));

        printf("%d\n",peek(&s));

        printf("%d\n",pop(&s));

        printf("%d\n",peek(&s));

}

*/



/*

#include<stdio.h>
#include<stdlib.h>


typedef struct
{
        int data[10];
        int top;
        int size;
}queue;
```

```
void initial(queue *q)
{
        q->top=0;
        q->size=0;
}


void enqueue(queue *q,int x)
{
        q->data[q->size++]=x;
}


int dequeue(queue *q)
{
        return q->data[q->top++];
}


int main()
{
        queue q;

        initial(&q);

        enqueue(&q,1000);

        enqueue(&q,2000);

        printf("%d\n",dequeue(&q));

        enqueue(&q,3000);

        enqueue(&q,4000);

        printf("%d\n",dequeue(&q));


        enqueue(&q,5000);

        enqueue(&q,6000);

        printf("%d\n",dequeue(&q));

        enqueue(&q,7000);

        enqueue(&q,8000);

        printf("%d\n",dequeue(&q));

        return 0;
}

*/
```

```
/*

#include<stdio.h>
#include<stdlib.h>

#define MAX 10

typedef struct                                          //queue
{
        int data[MAX];
        int f;
        int r;
}queue;


void initial(queue *q)
{
        q->f=-1;
        q->r=-1;
}


int full(queue *q)
{
        return q->r == MAX-1;
}


int empty(queue *q)
{
        return q->r == -1 || q->f > q->r;
}


void enqueue(queue *q,int x)
{
        if(full(q))
        printf("Overload\n");
        else
        {
                q->data[++q->r]=x;
                if(q->f==-1)
                q->f=0;
        }
}
```

```
int dequeue(queue *q)
{
        if(empty(q))
        {
                printf("Empty!\n");
                return -1;
        }
        else
        {
                int z = q->data[q->f++];
                if(q->f > q->r)
                q->f=q->r=-1;
                return z;
        }
}


int peek(queue *q)
{
        if(empty(q))
        {
                printf("Empty!\n");
                return -1;
        }
        else
        {
                return q->data[q->f++];
        }
}


int main()
{
        queue q;

        initial(&q);

        printf("%d\n",peek(&q));

        enqueue(&q,1000);

        enqueue(&q,2000);

        printf("%d\n",dequeue(&q));

        enqueue(&q,3000);

        enqueue(&q,4000);

        printf("%d\n",dequeue(&q));


        enqueue(&q,5000);
```

```
        enqueue(&q,6000);

        printf("%d\n",dequeue(&q));

        enqueue(&q,7000);

        enqueue(&q,8000);

        printf("%d\n",dequeue(&q));

        enqueue(&q,9000);

        enqueue(&q,10000);

        printf("%d\n",dequeue(&q));

        enqueue(&q,11000);

        printf("%d\n",dequeue(&q));

        return 0;

}

*/




/*

#include<stdio.h>
#include<stdlib.h>

typedef struct node                             //Singly Linked List
{
        int data;
        struct node *next;
}node;


void insertbegin(node **ohead,int x)
{
        node *new = (node *)malloc(sizeof(node));

        new->data = x;

        new->next = (*ohead);

        (*ohead) = new;

}
```

```
void insertend(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        node *p = (node *)malloc(sizeof(node));

        p = *o;

        new->data = x;
        new->next = NULL;

        while(p->next!=NULL)
        p = p->next;

        p->next = new;
}


void show(node *h)
{
        while(h!=NULL)
        {
                printf("%d ",h->data);
                h = h->next;
        }
        printf("\n");
}


int main()
{
        node *head = NULL;

        insertbegin(&head,10);

        insertbegin(&head,20);

        insertbegin(&head,30);

        show(head);

        insertend(&head,40);

        insertend(&head,50);

        insertend(&head,60);

        show(head);

}

*/
```

```
/*

#include<stdio.h>
#include<stdlib.h>


typedef struct node
{
        int d;
        struct node *n;
}node;


void insertbegin(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        new->d = x;

        new->n = *o;

        *o = new;

}


void insertend(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        node *t = *o;

        while(t->n!=NULL)
        t = t->n;

        new->d = x;
        new->n = NULL;

        t->n = new;

}



void show(node *o)
{
        while(o!=NULL)
        {
                printf("%d ",o->d);
                o = o->n;
        }
```

```
        printf("\n");
}



int main()
{
        node *a = NULL;

        insertbegin(&a,10);

        insertbegin(&a,20);

        insertbegin(&a,30);

        show(a);


        insertend(&a,40);

        insertend(&a,50);

        insertend(&a,60);

        show(a);


}

*/




/*

#include<stdio.h>
#include<stdlib.h>

typedef struct node                                                    //
left right list
{
        int d;
        struct node *r;
        struct node *l;
}node;


void insertbegin(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        new->d = x;
```

```
        new->r = *o;
        new->l = NULL;

        if(*o != NULL)
        (*o)->l = new;

        *o = new;

}


void insertend(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        node *t = *o;

        while(t->r != NULL)
        t = t->r;

        t->r = new;

        new->l = t;
        new->d = x;
        new->r = NULL;

}


void show(node *o)
{
        while(o!= NULL)
        {
                printf("%d ",o->d);
                o = o->r;
        }
        printf("\n");
}


int main()
{
        node *a = NULL;

        insertbegin(&a,10);

        insertbegin(&a,20);

        insertbegin(&a,30);

        show(a);


        insertend(&a,40);
```

```
        insertend(&a,50);

        insertend(&a,60);

        show(a);


}

*/









/*

#include<stdio.h>
#include<stdlib.h>

typedef struct node
{
        int d;
        struct node *l;
        struct node *r;
}node;


void insertbegin(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        new->d = x;

        new->r = *o;

        *o = new;
}


void insertend(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));
```

```
        node *t = *o;

        while(t->r != NULL)
        t = t->r;

        t->r = new;

        new->l = t;
        new->d = x;
        new->r = NULL;

}


void show(node *o)
{
        while(o != NULL)
        {
                printf("%d ",o->d);
                o = o->r;
        }
        printf("\n");
}


int main()
{
        node *a = NULL;

        insertbegin(&a,10);

        insertbegin(&a,20);

        insertbegin(&a,30);

        show(a);


        insertend(&a,40);

        insertend(&a,50);

        insertend(&a,60);

        show(a);


}

*/
```

```
/*

#include<stdio.h>
#include<stdlib.h>


typedef struct node
{
        int d;
        struct node *l;
        struct node *r;
}node;


void insertbegin(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        new->d = x;

        new->r = *o;

        *o = new;
}


void insertend(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        node *t = *o;

        while(t->r != NULL)
        t = t->r;

        t->r = new;

        new->l = t;
        new->d = x;
        new->r = NULL;

}


void delete(node **o)
{
        int z;
        node *t=*o,*x,*y;

        printf("z : ");
        scanf("%d",&z);
```

```
        while(t->d!=z)
        t=t->r;

        x=t->l,y=t->r;

        x->r=y,y->l=x;

}

void show(node *o)
{
        while(o != NULL)
        {
                printf("%d ",o->d);
                o = o->r;
        }
        printf("\n");
}


int main()
{
        int n,z;
        node *a = NULL;

        printf("n : ");
        scanf("%d",&n);

        printf("ele : ");
        scanf("%d",&z);
        insertbegin(&a,1);

        for(int i=0;i<n-1;i++)
        {
                scanf("%d",&z);
                insertend(&a,z);
        }

        show(a);
        delete(&a);
        show(a);

}

*/
```

```
/*

#include<stdio.h>
#include<stdlib.h>


typedef struct node
{
        int d;
        struct node *n;
}node;


void insertbegin(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        new->d = x;

        new->n = *o;

        *o = new;

}


void insertend(node **o,int x)
{
        node *new = (node *)malloc(sizeof(node));

        node *t = *o;

        while(t->n!=NULL)
        t = t->n;

        new->d = x;
        new->n = NULL;

        t->n = new;

}


int count(node *o)
{
        int c=0;
        node *t=o;

        while(t->n!=NULL)
        {
```

```
                c++;
                t = t->n;
        }c++;

        printf("Count : %d\n",c);

        return c;
}


void deletemid(node **o)
{
        int c=count(*o);
        node *t = *o,*u,*v;

        c/=2;

        for(int i=1;i<c;i++)
        t = t->n;

        u = t->n;
        v = u->n;
        t->n = v;

}


void search(node *o)
{
        node *t=o;
        int z,c,d=0;

        printf("search : ");
        scanf("%d",&z);

        c=count(o);

        while(o->d!=z)
        {
                o=o->n;
                d++;
        }d++;

        if(c==d)
        printf("not found\n");
        else
        printf("found %d\n",o->d);

}


void rotate(node **o);
```

```
void show(node *o)
{
        while(o!=NULL)
        {
                printf("%d ",o->d);
                o = o->n;
        }
        printf("\n");

}



int main()
{
        int n,x;

        node *a = NULL;

        printf("n : ");
        scanf("%d",&n);

        printf("ele : ");
        scanf("%d",&x);
        insertbegin(&a,x);
        for(int i=0;i<n-1;i++)
        {
                scanf("%d",&x);
                insertend(&a,x);
        }

        show(a);
        rotate(&a);

//      show(a);
//      search(a);

//      show(a);
//      deletemid(&a);
//      show(a);
//      deletemid(&a);

        show(a);

}




void rotate(node **o)
{
        int c=count(*o);
        node *t = *o,*u = *o;
```

```
        while(t->n!=NULL)
        t=t->n;

        for(int i=0;i<c;i++)
        {
                t->n = u;
                u = u->n;
                t = t->n;
        }
        t->n=NULL;

}

*/



/*


#include<stdio.h>
#include<stdlib.h>


typedef struct dll
{
        int d;
        struct dll *l;
        struct dll *r;
}dll;


void insertbegin(dll **o, int x)
{
        dll *new = (dll *)malloc(sizeof(dll));
        if(new==NULL)
        return;

        new->d = x;

        new->l = NULL;

        new->r = *o;

        *o = new;

}


void insertend(dll **o, int x)
{
        dll *new = (dll *)malloc(sizeof(dll));
        if(new==NULL)
```

```
        return;

        dll *t = *o;

        new->d = x;

        new->r = NULL;

        while(t->r!=NULL)
        t=t->r;

        new->l = t;

        t->r = new;

}


int count(dll *o)
{
        int c=0;

        while(o->r!=NULL)
        {
                o=o->r;
                c++;
        }c++;

        return c;
}


int search(dll *o)
{
        int c=0,y;

        printf("Search : ");
        scanf("%d",&y);

        while(o->d!=y)
        {
                o=o->r;
                c++;
        }c++;

        printf("Found %d\n",o->d);

        return c;
}


void delete(dll **o);
```

```
void show(dll *o)
{
        while(o!=NULL)
        {
                printf("%d ",o->d);
                o=o->r;
        }
        printf("\n");
}




int main()
{
        dll *a = NULL;
        int z,n,m;

        printf("n : ");
        scanf("%d",&n);

        m=n/2;
        n-=m;

        printf("ele : ");
        for(int i=0;i<m;i++)
        {
                scanf("%d",&z);
                insertbegin(&a,z);
        }

        for(int i=0;i<n;i++)
        {
                scanf("%d",&z);
                insertend(&a,z);
        }

        show(a);

        printf("Count : %d\n",count(a));

        delete(&a);

        printf("Count : %d\n",count(a));

//      printf("C : %d\n",search(a));

        show(a);

        return 0;

}
```

```
void delete(dll **o)
{
        dll *t = *o;
        int c = search(*o);

        for(int i=1;i<c;i++)
        t=t->r;

        if(t->l!=NULL)
        t->l->r = t->r;
        else
        *o = t->r;

        if(t->r!=NULL)
        t->r->l = t->l;

        free(t);

}


*/







/*

#include<stdio.h>
#include<stdlib.h>


typedef struct cnode                    //Cicular ll
{
        int d;
        struct cnode *n;

}cnode;


void insertb(cnode **o, int x)
{
        cnode *new = (cnode *)malloc(sizeof(cnode));
        cnode *t = *o;

        int z=o->d;
```

```
        new->d = x;

        if(o==NULL)
        new->n = *o;
        else
        {
                while(t->d!=z)
                t = t->n;

                new->n = *o;

                t-n

        }

        *o = new;
}



void inserte(cnode **o, int x)
{

        cnode *new = (cnode *)malloc(sizeof(cnode));
        cnode *t = *o;

        new->d = x;

        if(*o == NULL)
        {
                new->n = *o;
                *o = new;
        }
        else
        {
                while(t->n != *o)
                t = t->n;

                t->n = new;
                new->n = *o;
        }

}


void show(cnode *o)
{
        if(o==NULL)
        return;

        cnode *t = o;

        do
        {
```

```
                printf("%d ",t->d);
                t=t->n;
        }while(t != o);
        printf("\n");

}


int main()
{
        stackll *a = NULL;

        inserte(&a,10);
        inserte(&a,20);
        inserte(&a,30);

        show(a);

        return 0;
}


*/









/*

#include<stdio.h>
#include<stdlib.h>


typedef struct stackll                          //stack ll
{
        int d;
        struct stackll *n;

}stackll;


void push(stackll **o, int x)
{
        stackll *new = (stackll *)malloc(sizeof(stackll));

        if(new == NULL)
        {
                printf("Memory Failed!");
                return;
```

```
        }

        new->d = x;

        new->n = *o;

        *o = new;

        printf("Pushed 😘: %d\n",new->d);

}



void pop(stackll **o)
{

        if(*o == NULL)
        {
                printf("Stack Underflow!\n");
                return;

        }

        printf("Popped 😘: %d\n",(*o)->d);

        *o = (*o)->n;

}


void show(stackll *o)
{
        printf("Stack 😘: ");

        if(o == NULL)
        {
                printf("Empty!\n");
                return;

        }

        stackll *t = o;

        while(o != NULL)
        {
                printf("%d ",o->d);
                o = o->n;
        }
        printf("\n");

}
```

```
void peek(stackll *o)
{

        if(o == NULL)
        {
                printf("Stack Underflow!\n");
                return;

        }

        printf("Peeked 😘: %d\n",o->d);

}



int main()
{
        stackll *a = NULL;
        int n,z;

        show(a);

        printf("n : ");
        scanf("%d",&n);

        for(int i=0;i<n;i++)
        {
                scanf("%d",&z);
                push(&a,z);
        }

        show(a);

        peek(a);

        pop(&a);

        show(a);

        return 0;
}

*/




/*
```

```
#include<stdio.h>
#include<stdlib.h>


typedef struct queue
{
        int d;
        struct queue *n;

}queue;


void enqueue(queue **o, int x)
{

        queue *new = (queue *)malloc(sizeof(queue));

        if(new == NULL)
        {
                printf("Memory Failed!");
                return;
        }

        new->d = x;
        new->n = *o;

        *o = new;

        printf("Enqueued 😘: %d\n",new->d);

}


void dequeue(queue **o)
{
        if(*o == NULL)
        {
                printf("Queue underflow!");
                return;
        }

        printf("Dequeued 😘: %d\n",(*o)->d);

        queue *t = *o,*u = *o;

        while(t != NULL)
        {
                u=t;
                t = t->n;
        }

        u->n = NULL;
```

```
}


void show(queue *o)
{
        printf("Queue 😘: ");

        if(o == NULL)
        {
                printf("Empty!\n");
                return;
        }

        while(o != NULL)
        {
                printf("%d ",o->d);
                o = o->n;
        }
        printf("\n");

}


int main()
{
        queue *a = NULL;

        enqueue(&a,10);
        enqueue(&a,20);
        enqueue(&a,30);

        show(a);

        dequeue(&a);

        show(a);

        return 0;
}

*/
```